

8. TRANSACTIONS

Transaction [May-08, May-07]

A collection of operations that forms a single logical unit of work is called a transaction. It is a unit of programs execution that accesses & possibly updates various data items.

ACID properties: To ensure integrity of data, a transaction must possess the following properties, collectively called ACID properties. [Dec-06]

(i) **Atomicity:** Either all the operations of the x'action execute or none of its does.

(ii) **Consistency:** If it is assumed that the DB was in a consistent state before the x'action started then it must be in a consistent state when the x'action completes.

(iii) **Isolation:** Even though multiple x'actions are executing concurrently, each x'action is unaware of the existence of other x'actions.

(iv) **Durability:** Once a x'action completes successfully, the changes it has made to DB must persist, even if there are failures.

To ensure atomicity is responsibility of x'action manager of DBMS.

To ensure consistency is responsibility of application program who initiated the x'action.

To ensure Isolation is responsibility of concurrency control manager of DBMS.

To ensure durability is responsibility of recovery manager of DBMS.

It is assumed that the DB resides on the disk & x'action access the data using:

(i) read(x), which x'fers data item x from DB into the local buffer of the x'action.

(ii) write(x), which informs DBMS to x'for data item x from local buffer to the DB.

A x'action which x'fers Rs.500/- from account A to account B can be given as

Ti: read (A)

$A = A - 500$

Write (A)

Read (B)

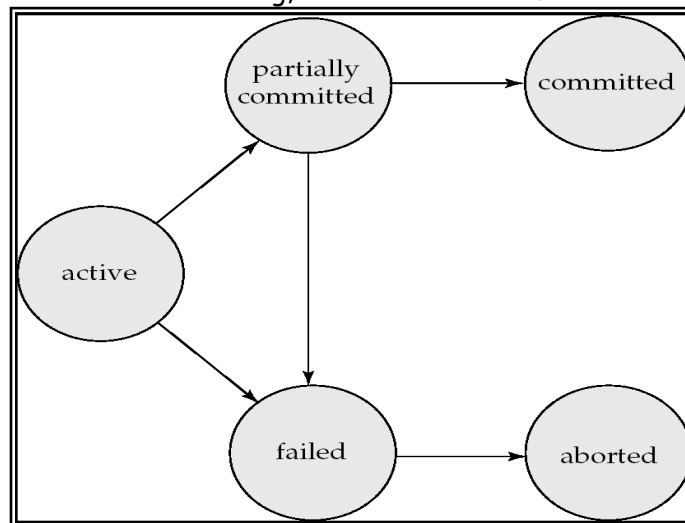
$B = B + 500$

Write (B)

Transaction states:

At given moment, a x'action is said to be in one of the five states.

(i) **active:** Initially when a x'action starts executing, it is in active state.



(ii) **partially committed:** When the last operation of a x'action is completed successfully it is said to enter partially committed state. All updates of x'actions may be still there in local buffer only.

(iii) **failed:** When it is decided that the x'action can not proceed with its normal execution, it enters failed state to ensure atomicity, all opn performed by the failed x'action must be undone & x'action is said to be rolled back.

(iv) aborted: Once a failed x'action is rolled back & DB is restored to its state prior to the start of the x'action, it enters aborted state. When a x'action is rolled back & it enters aborted state, the DBMS has two options:

(i) kill the x'action (ii) restart the x'action.

(v) Committed: Once a x'action completes all its operations & all its updates are reflected in the DB, it enters committed state. The operations of a committed x'action can not be undone. To nullify the effect of a committed x'actions, a compensating x'action must be written.

[Q: What is transaction? Discuss state transition diagram & properties of a transactions. May08]

[Q: What is transaction? Explain ACID properties. May08]

[Q: Explain Transaction Processing in Database System. May08]

Concurrency Control: In multiprogramming environment may x'actions may be executing concurrently. It is necessary for the system to control the interaction among the concurrent x'actions in order to prevent them from destroying the consistency of the DB. This control is achieved through a variety of mechanisms collectively known as concurrency control schemes. [May-08,Dec-06]

Schedule: A specified execution sequence is called schedule. It represents the chronological order in which instructions are executed in the system.

Serial schedule: It consists of a sequence of instructions from various x'actions where the instructions belonging to one single x'action appear together. For a set of n x'actions there exists n! different valid serial schedules.

For e.g. consider the following schedule.

To	T ₁
read(A) A:=A - 500 write(A) read(B) B:=B+500 write(B)	read(A) temp:=A*0.1 A:=A - temp write(A) read(B) B:=B+temp write(B)

Schedule 1

This is a serial schedule, where To followed by T1(<To,T1>) or T1 followed by To(<T1, To>) results in same state. If x'actions involved in a schedule themselves maintain consistency property then any serial schedule leaves DB in a consistent state.

Concurrent (Non Serial) schedule is the one where instructions from various x'actions involved in the schedule overlap in time.

To	T ₁
read(A) A:=A - 500 write(A)	read(A) temp:=A*0.1 A:=A - temp write(A)
read(B) B:=B+500 write(B)	read(B) B:=B+temp write(B)

Schedule 2

This is a concurrent schedule which leaves the DB in correct state as the one in which schedule 1 leaves.
But all concurrent schedules may not result in correct state.

To	T ₁
read(A) A:=A - 500 write (A) erad(B) B:=B+500 write(B)	read(A) temp:=A*0.1 A:=A - temp write(A) read(B) B:=B+temp write(B)

Schedule 3

This example indicates that the effect of executing a concurrent schedule must same as that of a serial schedule i.e. A concurrent schedule must be equivalent to a serial schedule; Requirement of this form is called serializability. [May-08]

Conflict serializability:

Consider a schedule S, in which there are two consecutive instructions I_i & I_j of x'actions T_i & T_j respectively. If they refer to different data items then we can interchange their execution sequence (swap them) without affecting the final result of the schedule.

But if they refer to the same data item Q then the order in which they execute may make difference.

(i) I_i = read(Q), I_j = read(Q): This order does not matter.

(ii) I_i = read(Q), I_j = write(Q): This order matters.

(iii) I_i = write(Q), I_j = read(Q): This order also matters.

(iv) I_i = write(Q), I_j = write(Q): This order matters if there are no subsequent writes on Q.

That is, two consecutive instructions from different x'actions said to conflict, if they operate on the same data item & at least one of the them is a write operation.

For e.g.

To	T ₁
read(A) write(A) read(B) write(B)	 read(A) write(A) read(B) write(B)

Conflicting instructions

Schedule 4

Let I_i & I_j be consecutive instructions of schedule S. If I_i & I_j are instructions of different x'actions & I_i & I_j do not conflict, then we can swap them to obtain a new schedule s' which will be equivalent to s.

For e.g. we can obtain a new schedule from schedule 4.

To	T ₁
read(A) write(A) read(B)	 read(A)

Schedule 5

write(B)	write(A) read(B) write(B)
----------	-------------------------------------

Schedule 5 is equivalent to schedule 4.

If a schedule S can be x'formed into a schedule S' by a series of swaps of non conflicting instructions, then schedules S & S' are called conflict equivalent.

A concurrent schedule S is **conflict serializable** if it is conflict equivalent to a serial schedule. That is, after a series of swaps it can be x'formed into a serial schedule.

For e.g. schedule 4 is conflict serializable.

Limitation: Certain concurrent schedules may have equivalent effect as some serial schedule, but they are not conflict serializable & rejected by the system.

For e.g.

To	T ₁
read(A) A:=A - 500 write(A) read(B) B:=B+500 write(B)	read(A) B:=B - 100 write(B) read(A) A:=A+500 write(A)

Schedule 6

Schedule 6 has the same effect as serial schedule $\langle T_0, T_1 \rangle$ or $\langle T_1, T_0 \rangle$ but it is not conflict serializable & so rejected by the system.

View serializability: [May-08]

Consider two schedules s & s' , where same set of x'actions participate in both the schedules. The schedules s & s' are said to be view equivalent, if the following three conditions are satisfied.

- for each data item Q , if x'action T_i reads initial value of Q in schedule s then in schedule s' also x'action T_i only reads the initial value of Q .
- for each data item Q , if x'action T_i reads value of Q produced by x'action T_j in schedule s , then in schedule s' also x'action T_i must read value of Q produced by x'action T_j .
- For each data item Q , if x'action T_i performs the final write (Q) in schedules, then in schedule s' also x'action T_i only perform the final write (Q).

In our previous examples schedule1 & schedule 2 are view equivalent.

A concurrent schedule s is said to view serializable if it is view equivalent to a serial schedule.

Advantage: Certain concurrent schedule, that are not conflict serializable are view serializable.

For e.g.

To	T ₁	T ₂
read(Q)		
write(Q)	write(Q)	
		write(Q)

Schedule 7

Schedule 7 is view serializable, since it is view equivalent to a serial schedule $\langle T_0, T_1, T_2 \rangle$ but it is not conflict serializable.

Every conflict serializable schedule is view serializable but not all view serializable schedules are conflict serializable.

The writes in schedule 7 applied by x'actions T_1 & T_2 are called blind writes & they appear in schedules that are view serializable but not conflict.

[Q: What do you understand by concurrent execution of transaction? Explain Conflict Serializability and view serializability. May08]

[Q: Explain serializability and serializable schedules. May08]

[Q: Explain conflict serializability and view serializability. May08]

[Q: Write short note on: Serializability. May08]

[Q: Explain Serializability with an example. May08]

Recoverability: If a x'action T_i fails. It is required to undo the effects of this x'action in order to maintain atomicity property of the x'action. Under concurrent environment, it is necessary ensure that any x'action T_j that is dependent on T_i is also aborted.

Recoverable schedule: [May-08, May-07]

consider the following schedule.

To	T_1
read(A)	read(A)
write(A)	
read(B)	

Schedule 8

If x'action T_1 is allowed to commit immediately after executing read(A) then T_1 commits before T_0 . Now, if T_0 fails just before commits, it is rolled back. Since T_1 is dependent on T_0 , it also must be rolled back. But T_1 has already committed & can not be rolled back.

Such schedules are called non-recoverable schedules.

A recoverable schedule is the one where, for each pair of x'actions T_i & T_j such that T_j reads a data item written by T_i then commit opⁿ of T_i appears before the commit opⁿ of T_j .

Cascadeless schedules: Even if a schedule is recoverable, to recover correctly from a failure of x'action T_i , we may have to roll back several x'actions.

T_0	T_1	T_2
read(A) write(A)	read(A) write(A)	read(A)

Schedule 9

In schedule 9, if x'action T_0 fails just before commit, x'action T_1 & T_2 are also rolled back. This is called cascading roll back.

A cascadeless schedule is the one where for each pair for x'actions T_i & T_j such that T_j reads a data item written by T_i , the commit of T_i appears before read opⁿ of T_j .

Every cascadeless schedule is recoverable.

[Q: What is a recoverable schedule? Why is recoverability of schedules desirable? Are there any circumstances under which it would be desirable to allow non-recoverable schedules? May08]